

The AI interprets. The code decides.

A language model is the only thing that can read handwriting or infer what a student was attempting. It is not a source of truth about whether one line follows from another. GapFinder separates those two capabilities at the architectural level, not by prompt engineering — which is why the product can accuse a specific line by name.

PAGE 2

The pipeline

Every stage, coloured by who holds authority over the output.

PAGE 3

The authority boundary

Decision by decision: model, code, or stored evidence.

PAGES 4–5

Grounding & personalisation

Retrieval, citations that are never generated, and the learner state that shapes the next call.

PAGES 6–7

Failure & observability

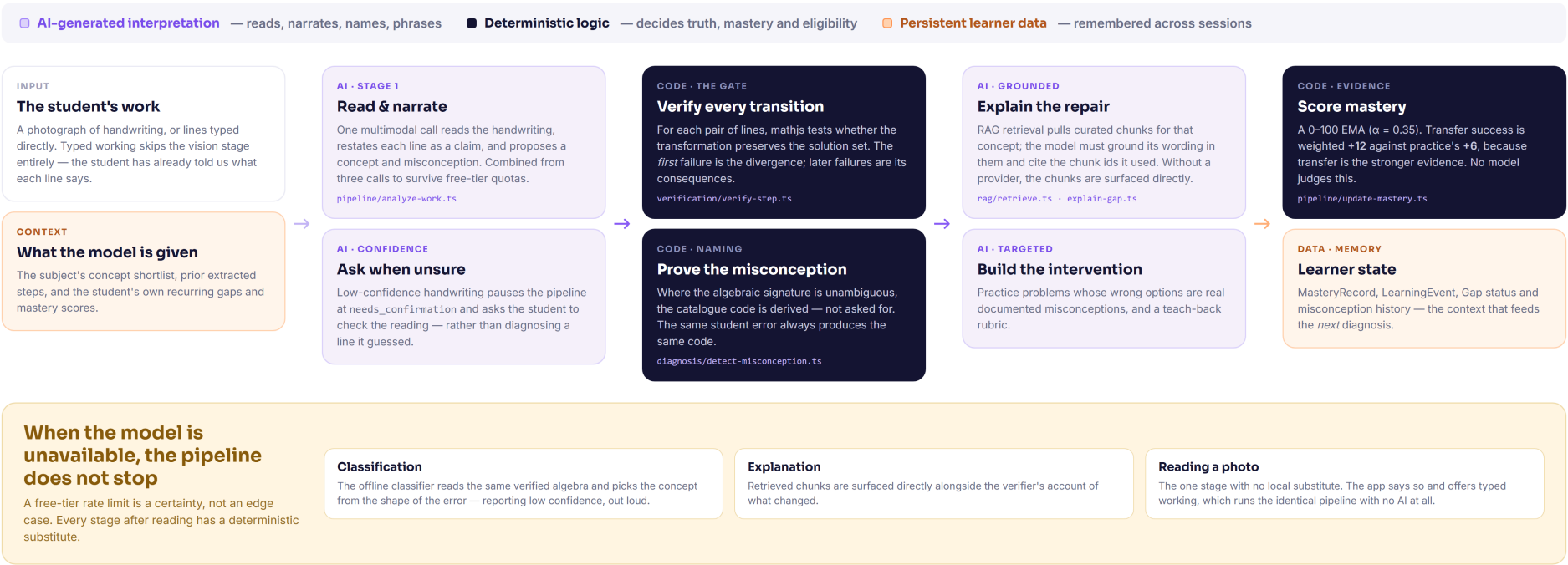
What happens when every provider is gone, and how to see what actually ran.

Colour marks **authority**, not technology. Nothing a model says about correctness reaches the student before deterministic code has confirmed it.

AI PIPELINE

The model interprets. The code decides.

Colour marks *authority*, not technology. Nothing the model says about correctness reaches the student until deterministic code has confirmed it.



Who is allowed to decide what

The test applied to every feature: *if the model returned nonsense here, would a student be told something false?* Where the answer is yes, the feature is built a different way.

AI-GENERATED INTERPRETATION Reads, narrates, names, phrases Reading, intent, concept slug, practice drafts, wording. Always <i>input</i> to the next stage — never a verdict.	DETERMINISTIC LOGIC Decides truth and eligibility Whether a step follows, where the divergence is, what the correction reads, and whether mastery may be claimed.	PERSISTENT LEARNER DATA Remembers across sessions Gaps, mastery EMAs, recurring codes, events, roadmap order. Written once at computation, read back thereafter.
--	---	--

DECISION	OWNER	WHY IT SITS THERE
What do the marks say, and what was the student attempting?	GEMINI VISION	Only a model can read handwriting or infer intent from an incomplete attempt. Low confidence pauses the pipeline and asks the student rather than guessing.
Does step n follow from $n-1$?	MATHJS	Must be provable. A model asked “is this step right?” is confidently wrong often enough to be useless for an accusation.
Where is the first divergence, and which later lines are its consequences?	SOLUTION-AUDIT.TS	The product's core claim, computed and measured by the eval harness. A line faithful to a wrong premise is not a new mistake.
What should that step have read?	SOLVE-STEP.TS	Derived from the student's own opening line. Never generated — a generated “correct” line could be wrong.
Which misconception is it?	SIGNATURE or CATALOGUE PICK	An unambiguous algebraic signature proves the code. Otherwise a model picks from the catalogue and it is labelled matched .
Which concept broke?	GEMINI → structural fallback	Needs judgement. With no provider, the offline classifier picks from the shape of the verified error.
Is a generated practice problem valid?	SOLVED INDEPENDENTLY	An unverified problem is never displayed.
Has the student mastered this?	EXAM/VERDICT.TS	Rules, not opinion: two questions minimum, sound reasoning required, a returning code decisive.

What the model receives, and what it returns

Stage boundaries are written to `analysis.status` as they are entered. The Analyzing screen polls that, so the progress a student watches is the real pipeline position rather than an animation on a timer.

WHAT IS SENT

The prompt payload

The work	A client-side downscaled and compressed photograph, or the student's typed lines
Subject	Chosen at capture; narrows the concept shortlist
Concept shortlist	The seeded slugs for that subject, so the model selects rather than invents
Learner context	Recurring misconception codes and mastery scores for the concepts in play
Output schema	A Zod schema reduced to the subset Gemini's structured-output endpoint accepts

WHAT COMES BACK

Structured, never prose to be parsed

Extracted steps	Each line as read, with a per-line confidence and a <code>needsConfirm</code> flag
Reasoning claims	Each line restated as what the student was asserting
Proposed concept	One slug from the shortlist — checked against the catalogue before use
Proposed code	A misconception code, discarded if it is not in the catalogue
Explanation	Grounded in retrieved chunks, with the chunk ids it used recorded

Three calls became one. Reading, narrating and classifying were once separate requests — three round trips and three charges against a free-tier quota for a single photograph. They are now one multimodal call whose result is carried forward, so later stages never re-ask what they were just told. This is only safe *because* nothing the model says about correctness is trusted downstream.

Where the words come from, and why they are about *this* student

Two separate mechanisms, often conflated. Retrieval decides what is *said*; the learner record decides what is *selected* and how the next call is framed.

01 · RETRIEVAL WITHOUT A VECTOR DATABASE

58 curated chunks, scored locally

Explanations are grounded in KnowledgeChunk rows retrieved per concept and filtered by kind — explanation, misconception, teaching_strategy, worked_example, practice_pattern. Scoring is local TF-IDF-style keyword overlap. No embedding API, no vector store, no recurring cost — and a corpus small enough for a human to read end to end.

Asked by name	Asked about by name with nothing scoring, all of that concept's chunks are returned — everything filed under it is relevant by construction
Cited for an error	If nothing scores, nothing is cited. An unrelated chunk is worse than none

02 · RESEARCH SOURCES

A model is never asked for a citation

Titles, authors, DOIs and URLs come back from provider APIs — **OpenAlex**, **Crossref** and **arXiv**, all free and keyless — and carry their provenance. The research layer contains no prompt at all. A fabricated citation that looks correct is worse than no citation, because a student would cite it.

Query mapping	Each concept slug maps to terms the literature actually uses; a misconception's display name is prose and matches on filler words
Two gates	A result must carry a distinctive on-concept anchor <i>and</i> an education signal, or it is discarded
Silence is named	Timed out, nothing relevant, no anchors, subject unsupported — four messages, never a blank panel

03 · PERSONALISATION

What makes the next analysis different from the first

Into the prompt. Recurring misconception codes and mastery scores for the concepts in play are passed as context, so the model chooses against a background rather than from scratch.

Into the selection. Which intervention is offered — more practice, transfer, teach-back — is decided in code from the gap's status and the mastery record, not by asking a model what would be nice.

Into the verdict. Exam Mode is given the codes this student held *before* the exam. A resurfacing code is decisive against mastery whatever the score.

The pipeline keeps working when its dependencies do not

Every stage after reading a photograph has a deterministic substitute, so a provider outage degrades confidence rather than breaking the product — and the interface says which it is.

THE CASCADE

cache → Gemini → Groq → deterministic

Gemini stays first: its vision is the strongest available here and the prompts are tuned to it. Groq's free tier is separate from Google's, so a Gemini quota exhaustion does not stop the product working.

FAILURE	BEHAVIOUR
quota	No retry — a free-tier limit is not transient
no_key	No retry. Skip to the next provider
unsupported	No retry — e.g. vision from a text-only provider
network	Retry in-provider, backoff 300 ms × attempt
invalid_response	Retry — schema validation may be transient

THE FAILURE THE PRODUCT REFUSES

"Couldn't check" is never reported as "nothing wrong". If every line came back uncertain, the analysis fails honestly. If only some did, the result says so: *"2 lines couldn't be checked, but everything we could verify holds."*

WHEN BOTH PROVIDERS ARE GONE

Classification	The offline classifier reads the same verified algebra and picks the concept from the shape of the error — reporting low confidence, out loud
Explanation	Retrieved chunks are surfaced directly alongside the verifier's account of what changed
Practice	Generated from templates, each solved independently before display
Verification	Untouched. It never used a model in the first place

THE ONE STAGE WITH NO SUBSTITUTE

Reading a photograph. Nothing local turns pixels into equations. The failure returns a `TYPE_INSTEAD`: prefix and the app offers typed working, which runs the *identical* pipeline with no AI involved at all.

FAILURES NAME THE PATH THAT STILL WORKS

QUESTION_ONLY:	A photographed question with no attempt — routed to guided solving, not a dead end
TYPE_INSTEAD:	Image reading failed — typing gets the same diagnosis

What actually ran, and what it cost

Every model call is recorded, so the pipeline can always account for itself — which provider served each stage, how long it took, and what it retrieved.

AI OBSERVABILITY · /dev/observability

Every model call is inspectable after the fact

AiUsageLog records, per call: the pipeline stage, provider:model, whether it was served from cache, whether it succeeded, latency, the error text when it did not, and the RAG chunk ids that fed the prompt. A per-analysis trace view replays the sequence.

This makes two things visible that are usually invisible: a **failover** — Gemini declined, Groq answered — and a **fallback**, where no provider answered and deterministic logic produced the result. Retrieval becomes traceable rather than assumed.

QUOTA MANAGEMENT

Free tiers are the design constraint

Response cache	AiCallCache, keyed by a content hash, 7-day TTL. A hit is logged as its own provider
Small keys	The key hashes the prompt, the system instruction and a digest of the image — not the image bytes
Request collapsing	Three pipeline calls became one multimodal call
Structured output	JSON schema on both providers — no reparse loops
Resource cache	14 days; a concept's literature changes monthly, not hourly
No regeneration	Opening a page again never re-runs an analysis

WHAT THE EFFICIENCY WORK BOUGHT

Round trips	Four sequential calls collapsed into one multimodal call, carried forward so no stage re-asks	Perceived latency	waitUntil returns the response before background persistence finishes; resources load after the diagnosis renders
Bytes on the wire	Photographs are downscaled and compressed in the browser before transmission	Repeat cost	Content-hashed caching, and a stored analysis is never regenerated — a revisited page costs nothing